

GOLDMAN SACHS DATA ANALYST INTERVIEW QUESTIONS

EXPERIENCE : 0-3 YOE

CTC : 23 LPA

DATA VISUALIZATION AND BI TOOLS

◆ 1. How would you visualize time-series data in Power BI/Tableau?

Time-series data refers to data points recorded at successive time intervals. In Power BI/Tableau, you can visualize time-series data using:

- ✓ **Line Chart** – Best for trends over time.
- ✓ **Area Chart** – Similar to a line chart but highlights the area below the line.
- ✓ **Bar Chart** – Useful for showing changes over discrete time periods.
- ✓ **Scatter Plot** – Helps in identifying patterns or anomalies over time.

Power BI Solution:

1. Import the dataset into Power BI.
2. Drag a **Date/Time column** to the X-axis of a **Line Chart**.
3. Drag the **Metric** (e.g., Sales, Revenue) to the Y-axis.
4. Apply a **Date Hierarchy** to drill down (Year, Quarter, Month, Day).
5. Use **Moving Averages** to smooth trends.

Tableau Solution:

1. Load the dataset and connect it to Tableau.

2. Drag the **Date field** to the Columns shelf.
 3. Drag the **Measure (e.g., Profit, Sales)** to the Rows shelf.
 4. Change the visualization to a **Line Chart**.
 5. Enable **Trend Lines** or **Forecasting** for deeper insights.
-

♦ 2. What are the different types of joins available in Power BI?

In **Power BI**, joins are performed using relationships between tables in **Power Query** or **Data Model**. The key types of joins are:

- ✓ **Inner Join** – Returns only matching records from both tables.
- ✓ **Left Outer Join** – Returns all records from the left table and matching records from the right.
- ✓ **Right Outer Join** – Returns all records from the right table and matching records from the left.
- ✓ **Full Outer Join** – Returns all records from both tables.
- ✓ **Anti Join (Left/Right Anti Join)** – Returns unmatched records from one table.

Example in Power Query (Power BI):

1. Open **Power Query Editor** → Click **Merge Queries**.
 2. Select the two tables to join.
 3. Choose a **common key** (e.g., Customer ID).
 4. Select the type of join (Inner, Left Outer, etc.).
 5. Expand the merged table to include required columns.
-

♦ 3. Explain the difference between a heatmap and a scatter plot.

- ✓ **Heatmap:**
 - A **heatmap** uses color gradients to represent data density or intensity.
 - Best for comparing multiple categories (e.g., Sales performance across regions and months).

- Typically used in **Power BI** with **Conditional Formatting** or in **Tableau** using **Density Marks**.

✓ **Scatter Plot:**

- A **scatter plot** represents individual data points using X and Y coordinates.
- Used to show **correlation** between two numerical variables (e.g., Revenue vs. Advertising Spend).
- In Power BI, it is created using the **Scatter Chart** visualization.

◆ **4. How would you create a dashboard to track key business KPIs?**

A **dashboard** provides a real-time visual representation of business performance. Steps to create a KPI dashboard in **Power BI/Tableau**:

Power BI Approach:

1. Load data and create relationships in **Power Query**.
2. Use **Card Visuals** to display key KPIs (e.g., Total Sales, Profit Margin).
3. Use **Bar/Column Charts** for categorical comparisons.
4. Use **Line Charts** for trends over time.
5. Apply **Slicers & Filters** to allow users to interact with the data.
6. Publish the report to **Power BI Service** and set up **scheduled refreshes**.

Tableau Approach:

1. Connect to the data source in **Tableau Desktop**.
 2. Create **individual sheets** for different KPIs.
 3. Use **Calculated Fields** to derive important metrics.
 4. Arrange charts and KPIs in a **Dashboard Layout**.
 5. Add **Filters & Actions** for interactivity.
 6. Publish to **Tableau Server** or **Tableau Public** for sharing.
-

◆ 5. What are calculated fields in Tableau/Power BI?

✓ **Calculated Fields** allow you to create new values from existing data using formulas.

Power BI (DAX) Example:

- **Creating a new column:**

$\text{SalesGrowth} = [\text{Current Year Sales}] - [\text{Previous Year Sales}]$

- **Creating a measure for Profit Margin:**

$\text{ProfitMargin} = \text{DIVIDE}([\text{Profit}], [\text{Revenue}])$

Tableau (Calculated Field) Example:

- **Creating a Profit Ratio:**

$[\text{Profit}] / [\text{Sales}]$

- **Conditional Sales Category:**

$\text{IF } [\text{Sales}] > 10000 \text{ THEN "High" ELSE "Low" END}$

STATISTICS AND PROBABILITY

◆ 1. What is the Central Limit Theorem (CLT), and why is it important?

✓ **Definition:**

The **Central Limit Theorem (CLT)** states that the distribution of the sample mean of a sufficiently large number of independent and identically distributed (i.i.d) random variables approaches a **normal distribution**, regardless of the original distribution of the population.

✓ **Formula:**

If $X_1, X_2, \dots, X_n$ are i.i.d. random variables with mean μ and variance σ^2 , then the sample mean:

$$\bar{X} = (X_1 + X_2 + \dots + X_n) / n$$

follows a **normal distribution** as $n \rightarrow \infty$.

✓ **Importance of CLT:**

- Enables the use of **normal distribution** for hypothesis testing and confidence intervals.

- Justifies why **sample means** are normally distributed, even when the population is not.
- Used in **statistical inference** (e.g., t-tests, confidence intervals).

✓ **Example:**

- Suppose we take **random samples** of size 50 from a skewed population (e.g., income data). According to the CLT, the **distribution of sample means** will be approximately normal.

♦ 2. How do you check if a dataset is normally distributed?

✓ **Methods to check normality:**

1 .Histogram & Density Plot:

- Plot a histogram of the data.
- If it follows a **bell-shaped curve**, it might be normally distributed.

2 .Q-Q Plot (Quantile-Quantile Plot):

- If the points lie **approximately on a straight line**, the data is normal.

3 .Shapiro-Wilk Test:

- Hypothesis:
 - H_0 : Data follows a normal distribution.
 - H_1 : Data does not follow a normal distribution.
- If the **p-value** < **0.05**, reject H_0 (not normal).

from scipy.stats import shapiro

data = [12, 15, 14, 10, 9, 18, 20, 16, 13]

stat, p = shapiro(data)

print("Shapiro-Wilk Test: p-value =", p)

4 .Anderson-Darling Test:

- Another test to check normality; similar to Shapiro-Wilk but more powerful for larger datasets.

5 .Skewness & Kurtosis:

- **Skewness** measures asymmetry (should be close to 0 for normal distribution).
- **Kurtosis** measures the tail thickness (should be close to 3 for normal distribution).

from scipy.stats import kurtosis, skew

print("Skewness:", skew(data))

print("Kurtosis:", kurtosis(data))

✓ Conclusion:

- If most tests indicate normality, we assume the data follows a normal distribution.
- If not, consider **transformations** (log, square root) to normalize data.

◆ 3. What is the difference between correlation and covariance?

Feature	Correlation	Covariance
Definition	Measures the strength and direction of a relationship between two variables.	Measures how two variables change together .
Formula	$\rho_{X,Y} = \text{Cov}(X,Y) / (\sigma_X \sigma_Y)$	$\text{Cov}(X,Y) = E[(X - \mu_X)(Y - \mu_Y)]$
Range	Between -1 and +1	Can take any value
Scale Dependence	Independent of units	Depends on units
Interpretation	1 = perfect positive, 0 = no correlation, -1 = perfect negative	Positive: Variables move together, Negative: Move in opposite directions

✓ Example:

Consider two stocks, **Stock A and Stock B**:

- **Covariance** tells us if their prices move together (positive) or in opposite directions (negative).

- **Correlation** tells us **how strongly** they move together (normalized between -1 and 1).

```
import numpy as np
```

```
A = [10, 12, 15, 18, 20] # Stock A prices
```

```
B = [8, 9, 14, 17, 19] # Stock B prices
```

```
# Covariance
```

```
cov_matrix = np.cov(A, B)
```

```
print("Covariance Matrix:\n", cov_matrix)
```

```
# Correlation
```

```
corr_matrix = np.corrcoef(A, B)
```

```
print("Correlation Matrix:\n", corr_matrix)
```

✓ Key Takeaway:

- **Use covariance** to check if two variables **move together**.
- **Use correlation** when you need to compare **strength of relationships**.

♦ 4. Explain Type I and Type II errors in hypothesis testing.

✓ Definition:

Error Type	Definition	Consequence
Type I Error (False Positive)	Rejecting a true null hypothesis (H_0).	False alarm (e.g., convicting an innocent person).
Type II Error (False Negative)	Failing to reject a false null hypothesis (H_0).	Missing a real effect (e.g., letting a guilty person go free).

✓ Example:

Consider a **COVID-19 Test**:

- **Type I Error:** Healthy person is diagnosed as COVID-positive (false positive).
- **Type II Error:** Infected person is diagnosed as COVID-negative (false negative).

✓ **Formula:**

- **Type I Error Probability = Significance level (α)** → Typically **0.05**.
- **Type II Error Probability = β** (depends on sample size, effect size).

✓ **Reducing Errors:**

- **Lower α** reduces **Type I error** but increases **Type II error**.
- **Increase sample size** to reduce **Type II error**.

◆ 5. How would you determine if a coin is biased based on 100 flips?

✓ **Approach:**

1 .Define Hypotheses:

- **H_0 (Null Hypothesis):** The coin is **fair** ($P(\text{Heads}) = 0.5$).
- **H_1 (Alternative Hypothesis):** The coin is **biased** ($P(\text{Heads}) \neq 0.5$).

2 .Collect Data:

- Flip the coin **100 times** and record **number of heads (X)**.

3 .Use a Binomial Test:

- If the coin is fair, **X follows a Binomial distribution:** $X \sim \text{Binomial}(n=100, p=0.5)$
- Calculate **p-value** using binomial probability:

```
from scipy.stats import binom_test
```

```
X = 60 # Number of heads observed
```

```
n = 100 # Total flips
```

```
p_value = binom_test(X, n, 0.5, alternative='two-sided')
```

```
print("P-value:", p_value)
```

4 .Interpret Results:

- If **p-value** < **0.05**, reject H_0 (coin is biased).
- If **p-value** $\geq$ **0.05**, fail to reject H_0 (no strong evidence of bias).

✓ **Conclusion:**

- A **small deviation** (e.g., 52 heads, 48 tails) may not be significant.
- A **large deviation** (e.g., 70 heads, 30 tails) is likely **statistically significant**.

PYTHON FOR DATA ANALYSIS

◆ 1. How would you read a large CSV file in Python efficiently?

✓ **Challenges of reading large CSV files:**

- High memory consumption
- Slow processing time

✓ **Efficient Methods:**

1 .Using chunksize in Pandas

- This reads the file in smaller chunks instead of loading everything into memory at once.

```
import pandas as pd
```

```
chunk_size = 100000 # Read 100,000 rows at a time
```

```
for chunk in pd.read_csv("large_file.csv", chunksize=chunk_size):
```

```
    process(chunk) # Process each chunk separately
```

2 .Using usecols to Load Specific Columns

- If you don't need all columns, load only the required ones.

```
df = pd.read_csv("large_file.csv", usecols=["column1", "column2"])
```

3 .Using dtype to Optimize Memory Usage

- Assigning correct data types reduces memory usage.

```
df = pd.read_csv("large_file.csv", dtype={"id": "int32", "price": "float32"})
```

4 .Using dask for Parallel Processing

- dask processes large datasets in parallel.

```
import dask.dataframe as dd
```

```
df = dd.read_csv("large_file.csv")
```

✓ Best Practice:

- Use chunksize for iteration
- Optimize memory with dtype
- Load only needed columns (usecols)

♦ 2. Explain the difference between apply(), map(), and vectorization in Pandas.

✓ map() (Element-wise, for Series only)

- Works only on a **single Pandas Series**.
- Used for simple transformations (e.g., applying a function to each value).

```
df["salary"] = df["salary"].map(lambda x: x * 1.1) # Increase salary by 10%
```

✓ apply() (Row/Column-wise, for DataFrames & Series)

- Works on both **Series & DataFrame**.
- Can apply a function to each row (axis=1) or column (axis=0).

```
df["new_column"] = df["existing_column"].apply(lambda x: x * 2)
```

```
df["total"] = df.apply(lambda row: row["salary"] + row["bonus"], axis=1) # Row-wise operation
```

✓ Vectorization (Fastest, using NumPy)

- Uses **NumPy operations**, which are much faster than loops or apply().

```
df["salary"] = df["salary"] * 1.1 # Vectorized approach (Best performance)
```

✓ Performance Comparison:

Method	Speed	Works on
map()	Fast	Pandas Series
apply()	Medium	DataFrame & Series
Vectorization	Fastest	NumPy & Pandas

✓ Conclusion:

- **Use vectorization** wherever possible (faster).
- **Use apply()** when row-wise logic is needed.
- **Use map()** for simple element-wise operations on a Series.

◆ 3. Given a dataset, how would you detect and handle missing values?

✓ Step 1: Detect Missing Values

import pandas as pd

df = pd.read_csv("data.csv")

print(df.isnull().sum()) # Count missing values in each column

✓ Step 2: Handle Missing Values

1 .Remove Missing Data

df.dropna(inplace=True) # Remove rows with missing values

2 .Fill with Mean/Median (for Numeric Columns)

df["salary"].fillna(df["salary"].mean(), inplace=True) # Replace with mean

df["salary"].fillna(df["salary"].median(), inplace=True) # Replace with median

3 .Fill with Mode (for Categorical Columns)

df["department"].fillna(df["department"].mode()[0], inplace=True)

4 .Forward/Backward Fill (For Time Series Data)

df.fillna(method="ffill", inplace=True) # Forward fill

```
df.fillna(method="bfill", inplace=True) # Backward fill
```

✅ **Best Practice:**

- **Drop missing values** if data loss is minimal.
- **Fill with mean/median/mode** if data loss is significant.
- **Use forward/backward fill** for time-series data.

♦ **4. How do you merge two Pandas DataFrames on multiple columns?**

✅ **Syntax:**

```
df_merged = df1.merge(df2, on=["column1", "column2"], how="inner")
```

✅ **Types of Joins:**

how	Parameter Description
inner	Returns only matching rows (default)
left	Keeps all rows from df1, matching from df2
right	Keeps all rows from df2, matching from df1
outer	Returns all rows from both DataFrames

how	Parameter Description
inner	Returns only matching rows (default)
left	Keeps all rows from df1, matching from df2
right	Keeps all rows from df2, matching from df1
outer	Returns all rows from both DataFrames

✅ **Example:**

```
import pandas as pd
```

```
df1 = pd.DataFrame({  
    "ID": [1, 2, 3],  
    "Name": ["Alice", "Bob", "Charlie"],  
    "Dept": ["HR", "IT", "Finance"]  
})
```

```
df2 = pd.DataFrame({
```

```
"ID": [1, 2, 4],  
"Dept": ["HR", "IT", "Marketing"],  
"Salary": [50000, 60000, 55000]  
})
```

Merge on multiple columns

```
df_merged = df1.merge(df2, on=["ID", "Dept"], how="inner")  
print(df_merged)
```

✅ **Best Practice:**

- Ensure **column names match** in both DataFrames.
- Use **how="left"** if you don't want to lose data from the main DataFrame.

♦ **5. Write a Python script to find outliers in a dataset using the IQR method.**

✅ **Interquartile Range (IQR) Formula:**

✅ **Interquartile Range (IQR) Formula:**

$$IQR = Q3 - Q1$$

$$Lower\ Bound = Q1 - 1.5 \times IQR$$

$$Upper\ Bound = Q3 + 1.5 \times IQR$$

✅ **Python Code:**

```
import numpy as np
```

```
import pandas as pd
```

```
# Sample dataset
```

```
data = {"Salary": [40000, 45000, 60000, 70000, 80000, 120000, 150000, 500000]}
```

```
df = pd.DataFrame(data)
```

```
# Compute Q1, Q3, and IQR
```

```
Q1 = df["Salary"].quantile(0.25)
```

```
Q3 = df["Salary"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
# Compute bounds
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
# Identify outliers
```

```
outliers = df[(df["Salary"] < lower_bound) | (df["Salary"] > upper_bound)]
```

```
print("Outliers:\n", outliers)
```

✅ **Best Practice:**

- **Use IQR** for skewed data.
- **Use Z-score** for normally distributed data.
- **Consider winsorization** (capping outliers) instead of removal if data loss is a concern.

SQL

♦ **1. Write an SQL query to find the second-highest salary from an employee table.**

Solution:

To find the second-highest salary, we can use various methods such as using LIMIT, subqueries, or window functions.

Approach 1: Using LIMIT and OFFSET (For MySQL and PostgreSQL)

```
SELECT salary  
FROM employees  
ORDER BY salary DESC  
LIMIT 1 OFFSET 1;
```

- This query orders the salaries in descending order and skips the highest salary using OFFSET 1, returning the second-highest salary.

Approach 2: Using SUBQUERY (Works on most databases)

```
SELECT MAX(salary) AS second_highest_salary  
FROM employees  
WHERE salary < (SELECT MAX(salary) FROM employees);
```

- This query first selects the highest salary using a subquery and then finds the maximum salary that is smaller than the highest salary, which is the second-highest salary.

♦ 2. How do you remove duplicates from a table without using DISTINCT?

Solution:

You can remove duplicates using a ROW_NUMBER() window function or a GROUP BY clause, depending on the use case.

Using ROW_NUMBER() (This works in SQL Server, PostgreSQL, MySQL 8.0+, etc.)

```
WITH ranked_rows AS (  
    SELECT *, ROW_NUMBER() OVER (PARTITION BY column_name ORDER BY  
    some_column) AS row_num  
    FROM table_name  
)  
DELETE FROM ranked_rows WHERE row_num > 1;
```

- This query assigns a row number to each row based on the partitioned column (column_name) and removes the duplicates where row_num > 1.

Using GROUP BY

```
SELECT column1, column2
```

```
FROM table_name
```

```
GROUP BY column1, column2;
```

- This query groups by the columns you want to remove duplicates for and returns distinct values for each group.

♦ 3. Write an SQL query to calculate the running total of sales per month.

Solution:

A running total (or cumulative sum) can be calculated using a window function like SUM() with an OVER() clause.

```
SELECT month, sales,
```

```
       SUM(sales) OVER (ORDER BY month) AS running_total
```

```
FROM sales_table;
```

- This query calculates the cumulative sum of sales month by month using SUM(sales) and orders the rows by month. The OVER() clause tells SQL to apply the SUM() function across the ordered rows to get the running total.

Example:

Month	Sales	Running Total
Jan 2023	1000	1000
Feb 2023	1500	2500
Mar 2023	2000	4500

◆ 4. Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.

Solution:

- **INNER JOIN**

- Combines rows from both tables where there is a match in both.
- Non-matching rows are excluded.

SELECT *

FROM table1

INNER JOIN table2 ON table1.id = table2.id;

- **LEFT JOIN (or LEFT OUTER JOIN)**

- Returns all rows from the left table and matching rows from the right table.
- If there is no match in the right table, the result will include NULL for columns from the right table.

SELECT *

FROM table1

LEFT JOIN table2 ON table1.id = table2.id;

- **RIGHT JOIN (or RIGHT OUTER JOIN)**

- Returns all rows from the right table and matching rows from the left table.
- If there is no match in the left table, the result will include NULL for columns from the left table.

SELECT *

FROM table1

RIGHT JOIN table2 ON table1.id = table2.id;

- **FULL JOIN (or FULL OUTER JOIN)**

- Returns all rows from both tables, with NULL for non-matching rows.

SELECT *

FROM table1

FULL JOIN table2 ON table1.id = table2.id;

Example:

ID	Name	Department
1	Alice	HR
2	Bob	IT

For a LEFT JOIN with a table that contains a department and salary column, it would return:

ID	Name	Department	Salary
1	Alice	HR	5000
2	Bob	IT	NULL

◆ **5. Given a transactions table, write a query to find the top 3 customers with the highest spending.**

Solution:

To find the top 3 customers with the highest spending, we can use GROUP BY to sum the spending for each customer and then order the result by the total spending in descending order.

```
SELECT customer_id, SUM(amount) AS total_spending
```

```
FROM transactions
```

```
GROUP BY customer_id
```

```
ORDER BY total_spending DESC
```

```
LIMIT 3;
```

- This query groups the transactions by customer_id, calculates the total spending for each customer, orders the results by total_spending in descending order, and limits the output to the top 3 customers.

Example:

Customer_ID	Total_Spending
101	5000
102	4500
103	4000

Summary of Join Types:

JOIN Type	Description
INNER JOIN	Only rows that match in both tables are returned.
LEFT JOIN	All rows from the left table and matching rows from the right table; non-matching rows have NULL values from the right table.
RIGHT JOIN	All rows from the right table and matching rows from the left table; non-matching rows have NULL values from the left table.
FULL JOIN	All rows from both tables; non-matching rows have NULL values in the columns where there is no match.